```
================================================================================
VOICE COMPANION — MASTER CONTEXT DOCUMENT
Product Vision, Architecture, Features & Implementation Plan
================================================================================
Last updated: March 2026
Status: Ideation + Scrappy Prototype (Whisper STT working)
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
SECTION 1: THE VISION
--------------------------------------------------------------------------------

The core idea is deceptively simple: what if you had every great philosopher in
the world in your pocket, and you could just talk to them about your day?

Not a chatbot. Not a productivity tool. A private, voice-first AI companion that
listens to your life — passively, naturally, without friction — and builds a deep
understanding of who you are, what you care about, and how your mind works over
time.

The key philosophical insight that drives everything:

  People already talk to themselves. They talk in the car, on walks, to friends,
  in their heads. What if that natural stream of thought was captured, understood,
  and reflected back intelligently — without requiring any deliberate "logging"?

This is NOT:
  - A habit tracker that nags you to log things
  - A journaling app where you stare at a blank text box
  - A cloud-based surveillance product owned by a corporation
  - An AI that tells you what to do

This IS:
  - A private philosopher that listens and reflects
  - A system that understands you better the more you talk
  - A knowledge graph of your own life, built passively
  - Open source, locally hosted, owned entirely by the user

The decision to act on any insight is always the user's. The system informs.
It never prescribes.

--------------------------------------------------------------------------------
SECTION 2: WHY NOW
--------------------------------------------------------------------------------

```

This idea is more plausible today than it was even 12 months ago because:

1. Whisper (OpenAI, open source) runs locally and produces high-quality STT.
   Already validated — a scrappy prototype has been built with this.

2. Small local LLMs (Phi-3 Mini, Mistral 7B) are now capable enough to do
   entity extraction, sentiment analysis, causal reasoning, and persona-driven
   conversation — without reaching a cloud API.

3. Pipecat (voice agent framework) handles the hard real-time voice problems:
   latency, turn-taking, interruption handling, STT→LLM→TTS orchestration.
   This is pluggable — any LLM can be swapped in.

4. Graph databases (Kuzu, Neo4j embedded) are mature, fast, and embeddable.
   Graphiti by Zep provides real-time graph memory built specifically for
   AI agents — no batch recomputation, temporally aware.

5. On-device inference is becoming real. Phi-3 Mini runs on a modern phone.
   The bottleneck of "can a small model do this?" is largely solved.

6. Nobody has assembled all of this together into an open source product
   aimed at personal life intelligence. The gap is real and specific.

SECTION 3: WHAT EXISTS (COMPETITIVE LANDSCAPE)

Rewind AI
  - Passive capture: YES
  - Cloud-based: YES (disqualifying)
  - Knowledge graph memory: NO
  - Open source: NO

Rosebud
  - Voice journal: YES
  - Connects entries to past: YES (basic)
  - Graph memory: NO — flat storage
  - Local/private: NO
  - Open source: NO

Reflect, Notion AI, Mem
  - Text-first, not voice-first
  - Cloud-based
  - No longitudinal pattern tracking
  - No graph memory

Graphiti by Zep
  - Graph memory architecture: YES (excellent)
  - Built for enterprise AI agents, not personal life tracking
  - No product UX around it
  - No voice input

THE GAP:
  Voice-first + knowledge graph memory + truly local/private + open source +
  longitudinal personal analytics with graph algorithms.

  Every existing product hits 1 or 2 of these. Nobody has all four.
  The combination doesn't exist as a product. That is the opportunity.

SECTION 4: INPUT LAYER

Zero friction is the product's core value proposition. The user never "logs"
anything deliberately. Data comes in through three routes:

ROUTE 1: VOICE (primary)
  - User speaks naturally — in the car, on a walk, lying in bed, anywhere
  - Continuous or on-demand capture modes
  - Whisper handles STT locally (already prototyped)
  - No wake word required in passive mode
  - Pipecat handles the real-time voice pipeline

ROUTE 2: PHOTO + CONTEXT
  - User takes a photo (meal, place, person, thing)
  - Vision model extracts description + context
  - User can optionally add a voice note on top
  - Converts to the same structured format as voice

ROUTE 3: TEXT / PASTE
  - Optional fallback
  - Lightweight — not the primary experience

All three routes produce a raw transcript or description, which flows into the
Processing Layer. The user experience is identical regardless of input route.

WHY PASSIVE CAPTURE WINS:
  Every habit tracker fails for the same reason: it requires the user to
  remember to log. Passive capture removes this failure mode entirely.
  Because the user speaks naturally, the data reflects what is actually

```
  happening in their life — not a curated, post-hoc summary.

--------------------------------------------------------------------------------
SECTION 5: PROCESSING LAYER
--------------------------------------------------------------------------------

After each input, a small local LLM reads the raw transcript and extracts
structured data. This is NOT a summary. It is a skeleton — the minimum
structured representation needed to build and query the graph.

WHAT GETS EXTRACTED:

  Entity extraction
    People (named and unnamed — "my manager" becomes a node)
    Places (home, office, gym, etc.)
    Objects (things that recur)
    Events (things that happened)

  Sentiment tagging
    Valence: positive / neutral / negative
    Arousal: calm / anxious / energised / depleted
    Applied per entry, timestamped

  Theme detection
    Recurring topics that emerge over time
    Work stress, sleep, relationships, health, goals, creativity
    Themes are not predefined — they emerge from patterns

  Causal relationship detection
    "Because of X, Y happened"
    "After doing X, I felt Y"
    This is the highest-value signal in the system
    What leads to what — the causal graph of your life

IMPLEMENTATION NOTES:
  - LLM of choice (pluggable) handles extraction
  - Output is always structured JSON matching the graph schema
  - Fast, cheap inference — these are not hard tasks for a small model
  - Runs locally, no API call required

--------------------------------------------------------------------------------
SECTION 6: MEMORY LAYER — THE KNOWLEDGE GRAPH (THE CORE)
--------------------------------------------------------------------------------

This is what makes the system architecturally different from every other
```

```
journaling or AI assistant product. This is the moat.

THE PROBLEM WITH NAIVE APPROACHES:
  Most AI memory systems do one of two things:
  1. Dump raw text into the context window — bloats, degrades, hits limits
  2. Basic RAG (retrieve similar chunks) — flat and dumb, no understanding
     of relationships or causality

THE GRAPH SOLUTION:
  Everything extracted from voice/photo input is stored as nodes and edges
  in a graph database. The raw transcript goes to cold storage (SQLite).
  The graph stays lean and queryable forever.

  When the AI needs context, it gets a relevant SUBGRAPH — not the user's
  entire history. Context window is never a constraint.

NODE TYPES:
  person        — people in the user's life (named or relationship-based)
  emotion       — specific emotional states
  theme         — recurring topics or domains
  event         — things that happened
  habit         — recurring behaviours (active or inactive)
  place         — locations
  decision      — choices made
  outcome       — results of decisions or habits

EDGE TYPES (relationships between nodes):
  caused_feeling       Event → Emotion
  connected_to         Emotion → Theme
  recurs_with          Theme → Person/Place/Habit
  led_to               Decision → Outcome
  correlated_with      Habit → Emotion
  happened_during      Event → TimeContext
  involves             Event → Person

ALL EDGES ARE TEMPORAL:
  Every edge has a timestamp. This means the graph is not just a static map —
  it is a timeline. You can query how relationships change over time.

EXAMPLE GRAPH STRUCTURE:
  Event:"bad meeting"   ──caused_feeling──▶   Emotion:"anxious"
  Emotion:"anxious"     ──connected_to──▶      Theme:"work_stress"
  Theme:"work_stress"   ──recurs_with──▶       Person:"manager"
  Habit:"no_run"        ──correlated_with──▶   Emotion:"low_mood"
  Decision:"quit_sugar" ──led_to──▶            Outcome:"better_sleep"
```

```
WHAT GRAPH ALGORITHMS GIVE YOU FOR FREE (no LLM required):

  Centrality (PageRank-style)
    "Work stress is the most connected node in your life right now."
    Surfaces what's dominating your mental landscape without you asking.

  Community detection
    Clusters of related nodes surface automatically.
    Relationships the user didn't consciously notice.

  Trend detection
    Which themes are growing over time? Which are fading?
    Temporal edges make this trivial to compute.

  Path finding
    "What chain of events led from X to Y?"
    Traces causal chains through the graph.

STORAGE ARCHITECTURE:
  Hot storage:  Graph DB (Kuzu embedded or Neo4j) — nodes, edges, metadata
  Cold storage: SQLite — raw timestamped transcripts, compressed
  The graph never bloats because it stores structure, not content.

IMPLEMENTATION:
  Primary: Graphiti by Zep (open source)
    - Real-time incremental graph updates (no batch recomputation)
    - Temporally aware by design
    - Query latency suitable for voice applications
    - Built for exactly this kind of agent memory use case

  Graph algorithms: NetworkX (Python)
    - Centrality, community detection, shortest path, trend analysis
    - Runs locally, computationally inexpensive on graph sizes expected

OBSIDIAN COMPARISON:
  Obsidian lets users manually build a knowledge graph of notes.
  This system builds that graph automatically, from voice, over time,
  with typed edges and temporal metadata. The visual output (Phase 3)
  looks like Obsidian — but the graph builds itself.

--------------------------------------------------------------------------------
SECTION 7: PERSONA LAYER
--------------------------------------------------------------------------------
```

```
The persona is the soul of the product. The LLM is pluggable (user chooses).
The feeling of the system is defined entirely by a carefully engineered
system prompt.

THE PHILOSOPHICAL DESIGN PRINCIPLE:
  The system never tells the user what to do.
  It reflects. It asks. It surfaces. It informs.
  The decision is always the user's.

  This is the difference between a tool that optimises you like a machine
  and a companion that helps you think more clearly.

THREE PERSONA MODES (user-configurable):

  Stoic Listener
    Never judges. Reflects back what it hears.
    Minimal verbal output — lets silence do the work.
    Speaks only when spoken to, or when a pattern is genuinely significant.
    Good for: daily decompression, emotional processing.

  Socratic Mode
    Asks one question that makes the user think harder.
    Never provides the answer. Only deepens the question.
    "You mentioned X twice — what do you think is driving that?"
    Good for: decision-making, untangling confusion.

  Pattern Analyst
    Surfaces what the graph has found.
    "I've noticed you mention anxiety every time you have back-to-back meetings."
    Informed by centrality and correlation data from the graph.
    Good for: weekly review, habit awareness.

PROACTIVE vs REACTIVE (key design decision, unresolved):
  Option A: Purely reactive — only speaks when the user initiates.
  Option B: Proactive — surfaces a significant pattern when it detects one.
            E.g. "You haven't mentioned sleep positively in 11 days."

  Instinct: Start reactive. Let the user build trust with the system first.
  Introduce proactive nudges in Phase 2 as an opt-in feature.

WHAT THE LLM RECEIVES:
  Not the user's full history.
  A relevant subgraph + the current conversation turn.
  Small, fast, focused. The graph does the heavy lifting.
```

```
--------------------------------------------------------------------------------
SECTION 8: OUTPUT LAYER
--------------------------------------------------------------------------------

Multiple output surfaces, all optional. The user configures what they want.

Voice response (real-time)
  TTS of user's choice
  Conversational loop via Pipecat
  The primary experience — you talk, it responds

Weekly digest (async)
  Surfaced once a week (or on demand)
  Most central theme of the week
  Biggest shift in sentiment vs previous week
  One Socratic question to sit with
  Delivered as voice or text

Visual graph explorer (Phase 3)
  Obsidian-style interactive node map
  User can explore their own knowledge graph
  Zoom into clusters, trace causal chains
  Filter by time period, node type, theme

Agent hooks (ecosystem layer)
  Any external agent can query the graph via API
  Habit tracker: "Was the user physically active today?" → graph query
  Nutrition: "What did the user eat this week?" → graph query
  Finance: "Did the user mention spending anxiety?" → graph query
  The graph is the source of truth. Apps are just views on top of it.

Photo log
  Visual timeline of captured photos with extracted context
  Connected to graph nodes (places, people, events)

--------------------------------------------------------------------------------
SECTION 9: PRIVACY LAYER
--------------------------------------------------------------------------------

Privacy is not a feature. It is the product.

The entire value proposition collapses if the user cannot trust that their
most honest, unfiltered thoughts are not being sold, mined, or surveilled.

PRINCIPLES:
```

```
  Local first — everything runs on device by default
  No data leaves the device unless the user explicitly opts in to sync
  Graph DB encrypted at rest
  Raw transcripts encrypted at rest
  User owns the full export (graph + transcripts)
  Open source — every line of code that touches user data is auditable

LLM CHOICE:
  The user selects their LLM. Options include:
  - Fully local (Ollama + Phi-3 Mini / Mistral 7B) — zero data leaves device
  - Self-hosted VPS — slightly larger models, still private
  - Private API (non-OpenAI, non-Google) — user's informed choice

  The system never defaults to a cloud API. Local is the default.

OPEN SOURCE STRATEGY:
  Full open source release (Phase 3)
  MIT or Apache 2.0 license
  Community can add integrations, personas, fine-tuned models
  The codebase becomes a platform, not just an app
  This is the cult moment — when builders can fork and extend it

--------------------------------------------------------------------------------
SECTION 10: TECH STACK
--------------------------------------------------------------------------------

Every component exists today. Nothing needs to be invented.
The assembly is the work.

VOICE PIPELINE:
  Pipecat              Voice agent orchestration framework
                       Handles real-time STT→LLM→TTS pipeline
                       Manages turn-taking, interruption, latency
  Whisper              Speech-to-text (open source, runs locally)
                       Already prototyped and validated
  TTS of choice        Pluggable — Kokoro, Piper, Coqui, etc.

LLM LAYER:
  Fully pluggable — user chooses their model
  Recommended local options:
    Phi-3 Mini         On-device (iPhone 15 Pro class hardware)
    Mistral 7B         Self-hosted, significantly better persona quality
    Llama 3 8B         Good balance of quality and speed
  The system prompt does the heavy lifting — a well-engineered prompt
  on a small model beats a lazy prompt on a large model.
```

```
GRAPH DATABASE:
  Kuzu                  Embedded graph DB — runs in-process, no server needed
                        Fast, lightweight, embeddable in mobile apps
  Neo4j                 Alternative for self-hosted deployments
                        More tooling, better visualisation ecosystem

GRAPH MEMORY:
  Graphiti (by Zep)     Open source graph memory for AI agents
                        Real-time incremental updates
                        Temporally aware
                        Designed for voice/agent applications

GRAPH ALGORITHMS:
  NetworkX (Python)     Centrality, community detection, path finding
                        Trend analysis over temporal edges
                        Runs locally, fast on expected graph sizes

COLD STORAGE:
  SQLite                Raw transcripts, timestamped, compressed
                        Simple, embedded, battle-tested
                        Full-text search available via FTS5 extension

MOBILE WRAPPER:
  React Native          Cross-platform (iOS + Android)
                        Thin shell — local inference runs underneath
  OR Flutter            Alternative — better performance for audio
  OR Progressive Web App  Fastest to prototype, less native feel

VISION (for photo input):
  MobileVLM / LLaVA     Small vision-language models that run locally
  OR API fallback        Optional, user opt-in only

--------------------------------------------------------------------------------
SECTION 11: BUILD PLAN — THREE PHASES
--------------------------------------------------------------------------------

PHASE 1: THE CORE LOOP (target: working prototype in a weekend)
  Goal: Something you can actually talk to that feels different.

  What to build:
    - Pipecat + Whisper integration (STT already done)
    - Local LLM connected via Ollama
    - System prompt defining the Stoic Listener persona
    - Basic SQLite storage of transcripts
```

```
    - Minimal graph schema (just nodes + edges, no algorithms yet)
    - One hardcoded weekly insight format

  Success criterion:
    Talk to it for 10 minutes. It knows what you said 5 minutes ago.
    It asks one good question. It doesn't tell you what to do.
    That moment of "holy shit it actually understands me" = Phase 1 complete.

  Stack for Phase 1:
    Whisper (done) + Pipecat + Ollama (Mistral 7B) + SQLite + basic Kuzu schema

PHASE 2: THE GRAPH (target: 4-6 weeks)
  Goal: The system starts feeling like it knows you across sessions.

  What to build:
    - Graphiti integration for persistent graph memory
    - Entity + sentiment + causal extraction pipeline
    - Graph algorithm layer (NetworkX — centrality, correlations)
    - Pattern surfacing ("I've noticed...")
    - Weekly digest generation
    - Proactive nudges (opt-in)
    - Photo input (vision model integration)
    - Encrypted storage (graph + transcripts)

  Success criterion:
    After 2 weeks of use, it surfaces a correlation the user hadn't
    consciously noticed. That moment = Phase 2 complete.

PHASE 3: THE ECOSYSTEM (target: 3-6 months)
  Goal: Open source release + community + cult following.

  What to build:
    - Visual graph explorer (Obsidian-style, built with D3 or Cytoscape)
    - Agent hook API (external apps can query the graph)
    - Plugin system (community-built integrations)
    - Mobile app (React Native or Flutter)
    - Self-hostable server option (for users who want a home server)
    - Documentation + onboarding
    - Open source release (GitHub)

  Success criterion:
    First 100 users who are obsessive Obsidian/journaling types.
    One tweet from a high-signal person that goes viral.
    Community starts building integrations.

```

```
--------------------------------------------------------------------------------
SECTION 12: GO-TO-MARKET — THE CULT FOLLOWING STRATEGY
--------------------------------------------------------------------------------

Target user for initial traction: TWO entry points.

ENTRY POINT 1: RICH FEW (high-agency early adopters)
  Founders, builders, obsessive self-improvers, philosophy nerds
  Already use Obsidian, already journal, already think about systems
  Care deeply about privacy — already running self-hosted tools
  Have the hardware to run it locally (iPhone 15 Pro, M-chip Mac)
  Have audiences — one tweet from the right person = 10k waitlist
  They'll stress-test it, find edge cases, evangelize loudly

ENTRY POINT 2: POOR MANY (access to tools previously unavailable)
  People who can't afford therapy, coaching, or premium apps
  Being given something better than what rich people pay thousands for
  This spreads differently — more emotional, more loyal
  The open source + local model means cost approaches zero

THE PRODUCT WORKS FOR BOTH WITHOUT COMPROMISE:
  Rich guy: runs on iPhone 15 Pro with Phi-3 Mini locally
  Anyone else: runs on budget Android through lightweight hosted model
  Same graph, same intelligence, different hardware

RECOGNITION MOMENT:
  The target user is someone who has thought about this problem themselves.
  They've wanted something like this. When they see it, they immediately get it.
  No explaining required.
  "This is what I always wanted" = the cult starts here.

THE OPEN SOURCE ANGLE IS THE MOAT:
  Not a weakness. The community adds integrations, personas, fine-tuned models.
  You become the platform. Nobody can take that away by copying the product.

--------------------------------------------------------------------------------
SECTION 13: KEY DESIGN DECISIONS (UNRESOLVED / TO DECIDE)
--------------------------------------------------------------------------------

1. PERSONA: What is the default persona?
   Options: Stoic / Socratic / Analyst
   Instinct: Start with Stoic. Less is more for first impression.

2. PROACTIVE vs REACTIVE: Does it speak unprompted?
   Instinct: Reactive by default, proactive as opt-in in Phase 2.
```

3. WAKE WORD vs PUSH-TO-TALK vs ALWAYS ON:
   Always-on is the dream but battery/privacy risk.
   Push-to-talk is safest for v1.
   Wake word is middle ground.
   Instinct: Push-to-talk for Phase 1, wake word option in Phase 2.

4. GRAPH VISIBILITY: Is the graph visible to the user?
   Option A: Backend intelligence only (user never sees it)
   Option B: Obsidian-style visual explorer (Phase 3 feature)
   Instinct: Both. Hidden in Phase 1-2, visual explorer in Phase 3.

5. LLM DEFAULT: What's the default recommendation?
   Instinct: Mistral 7B self-hosted for best persona quality,
   Phi-3 Mini for fully on-device (mobile-first users).

6. MOBILE vs DESKTOP FIRST:
   Instinct: Mobile first — voice is a mobile-native behaviour.
   Desktop companion app as secondary (always-on, richer screen).

SECTION 14: CONTEXT FOR AI CODING ASSISTANTS

If you are an AI assistant (Claude Code or similar) reading this document to help build the Voice Companion, here is what you need to know:

PROJECT NAME: Voice Companion (working title — "The Philosopher")

CURRENT STATE:
  - Whisper STT prototype exists and works
  - Everything else is design/architecture — needs to be built

IMMEDIATE PRIORITY (Phase 1):
  Wire Pipecat + Whisper + Ollama (local LLM) + basic persona system prompt.
  Get a conversational loop working. Add basic SQLite transcript storage.
  Then add the simplest possible graph schema (nodes + edges in Kuzu).

ARCHITECTURE PRINCIPLES:
  - Every component must be swappable (LLM, TTS, STT, graph DB)
  - Local first — no cloud defaults
  - Graph is the source of truth for memory, not raw text
  - The LLM receives subgraphs, not full history
  - Keep it simple — complexity lives in the graph layer, not the app layer

```
TECH CHOICES (stick to these unless there's a strong reason not to):
  Voice orchestration:  Pipecat
  STT:                  Whisper (already done)
  LLM:                  Ollama (Mistral 7B default, Phi-3 Mini for mobile)
  Graph DB:             Kuzu (embedded)
  Graph memory:         Graphiti by Zep
  Graph algorithms:     NetworkX
  Cold storage:         SQLite
  Mobile:               React Native (Phase 3)
  Language:             Python for backend, React Native for mobile

DO NOT:
  - Use OpenAI API as default (user explicitly does not want this)
  - Store raw transcripts in the graph (cold storage only)
  - Send data to any cloud without explicit user opt-in
  - Build UI before the core voice loop works

--------------------------------------------------------------------------------
END OF DOCUMENT
================================================================================
```